Welcome to the ADI Interview Prep Series: Week 1

 Fall 2025

ATTENDANCE FORM!

01 THE GOAL

 The purpose of the Leet's Get Coding is to provide a open space, resources, and guidance for practicing and developing interview prep skills.

ICEBREAKER!

 Name

 Year

 Major

 What do you hope to get out of Interview Prep?

Today's Hosts

<Ella>

<Ryan>

Overview

```
Week 1: Arrays + Hashing

Week 2: Two Pointers + Sliding Window

Week 3: Binary Search + Trees

Week 4: Stack + Linked List

Week 5: Heap + Priority Queue

Week 6: Graphs

Week 7: Dynamic Programming + Greedy
```

Schedule:

Tuesdays 8-9pm @ Schermerhorn 608

Saturdays 2-3pm @ Uris 330

The two weekly sessions will cover the same content for that week

1 Hour

Beginner/Introductory Problems (Easy/Mediums)

- Solve problems in pairs/tuples
- Quick Review of key concepts
- Time for more Q&A with any addition questions

TODAY'S PLAN: ARRAYS/HASHING

01

Icebreaker

Review of Hashing
[8 minutes]

02

Problem 1: Two Sum

[12 minutes]

03

Problem 2: Valid Anagram

[15 minutes]

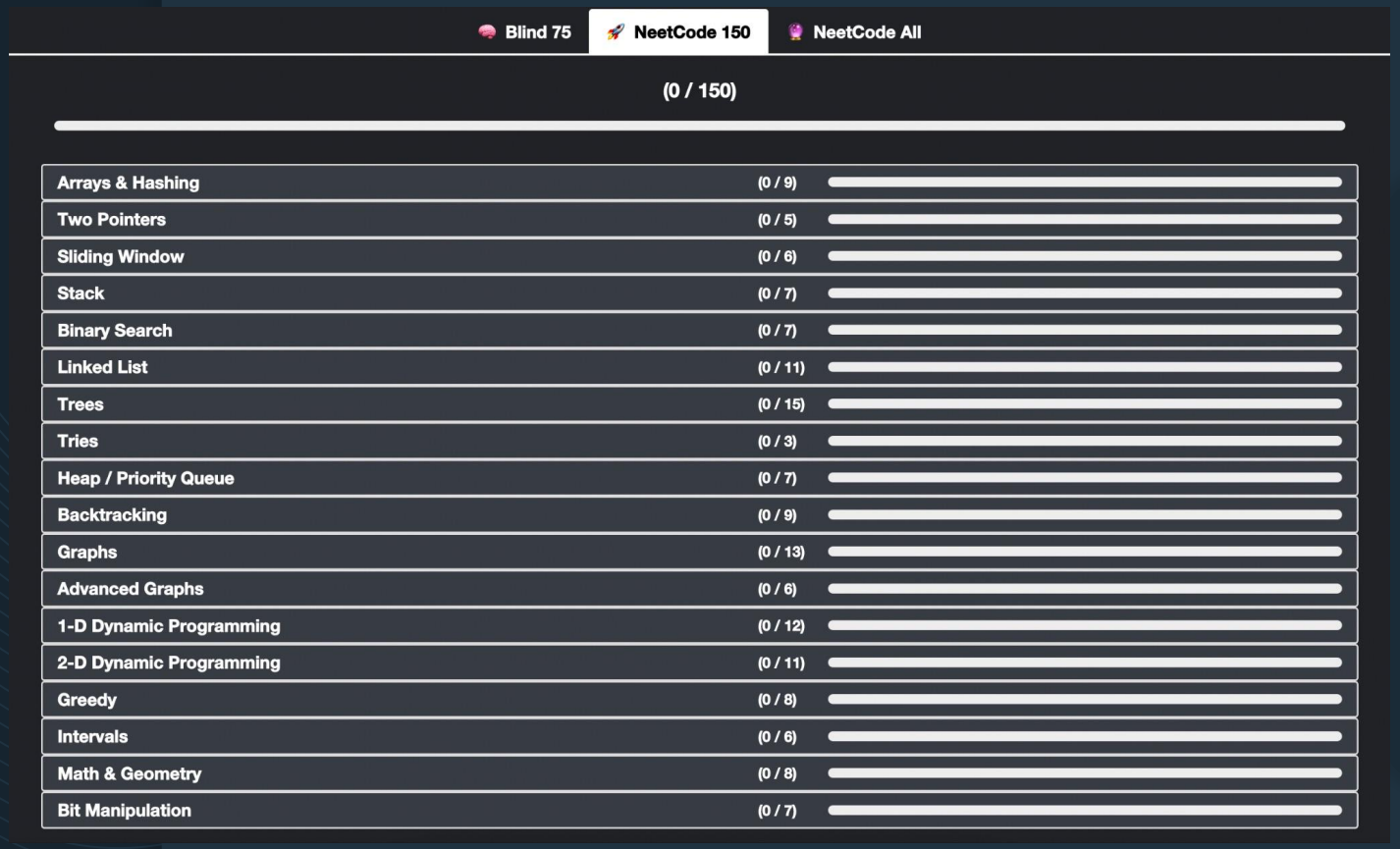
04

Problem 3: Longest Consecutive Sequence
[25 minutes]

05

06

Where to find the problems



https://neetcode.io/practice

LeetCode

https://leetcode.com/problemset/

Hash Maps

- Store key-value pairings
- O(1) average time complexity for insertions, deletions, and lookups

```
HashMap<K, V> map = new HashMap<K, V>();
```

```java
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {

        HashMap<String, Integer> map = new HashMap<>();

        // Insert elements using the put() method
        map.put("Alice", 16);
        map.put("Bob", 18);
        map.put("Sally", 20);

        // Retrieve values
        System.out.println("Bob's age: " + map.get("Bob"));

        // Check if a key exists
        System.out.println("Contains Alice? " + map.containsKey("Alice"));

        // Remove an entry
        map.remove( key: "Charlie");

        // Iterate through keys and values
        for (String key : map.keySet()) {
            System.out.println(key + " is " + map.get(key) + " years old.");
        }
    }
}
```

HashMap Methods

put (K key, V value)	Adds a key-value pair to the map	map.put("Alice", 25);
get (K key)	Returns value for a key (or null if not found)	map.get("Alice"); // 25
getOrDefault (K key, V defaultValue)	Returns value if key exists, otherwise returns the default value	map.getOrDefault("Bob", 0); // If "Bob" isn't there, returns 0
containsKey(K key)	Checks if a key exists in the map	map.containsKey("Alice"); // true
containsValue(V value)	Checks if a value exists in the map	map.containsValue(25); // true
remove(K key)	Removes a key-value pair	map.remove("Alice");
size ( )	Returns the number of key-value pairs	map.size(); // 3
isEmpty ( )	Checks if the map is empty	map.isEmpty(); // false
keySet( )	Returns a set of all keys	map.keySet();
values( )	Returns a collection of all values	map.values();
entrySet( )	Returns a set of key-value pairs	map.entrySet();

HashSet vs HashMap

Hash Set

- Stores unique elements. Does not store any key-value pairs; it only contains values.
- Can add an element, check if an element exists, and remove an element, all in average O(1) time
- In the solution to LCS, we use the set to quickly check if a number exists, which helps us efficiently find the start of a sequence and count its length.

Hash Map

- Stores data as key-value pairs. Each key in the map is unique, and it is associated with a specific value
- Average O(1) time complexity for inserting, accessing, and removing elements based on their keys
- In LCS solution, the keys are numbers from the array, and the values are the lengths of the consecutive sequences that the numbers are part of.

Pair Up!

 We are going to pair you up in groups of 2-3

- If you came with friends, feel free to work together
- For anyone without a partner we can help you find a pair!

 Don't spend more than 15-20 minutes per question

Try to talk out loud to work together with your partner(s) to solve the problem. In real interviews, being able to clearly communicate your solutions is super important! Some things to discuss could include

- Is there a brute force approach?
- Is there a more optimal approach?
- Are there trade offs with different data structures?
- Are there edge cases?
- etc…

PROBLEM ONE: TWO SUM

Given an array of integers, **nums**, and an integer, **target**, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: nums = [2,7,11,15], target = 9

Output: [0,1]

Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

PROBLEM ONE: TWO SUM

Given an array of integers, **nums**, and an integer, **target**, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

** Hint 1 (brute force):

Consider ways to iterate the numbers in the list.

E.g.

For num in list:

    [code]

PROBLEM ONE: TWO SUM

Given an array of integers, **nums**, and an integer, **target**, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

** Hint 2 (efficiency):

If you're using a loop, great! However, this is inefficient. What about a **hash map**?

Q: Why is the second solution more efficient than the first one? How do the run times differ?

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {

        int[] result = new int[2];

        for(int i = 0; i < nums.length; i++){
            int comp = target - nums[i];
            for(int j = i+1; j <= nums.length-1; j++)
            {
                if(nums[j] == comp){
                    result[0] = i;
                    result[1] = j;
                    return result;
                }
            }
        }
        return result;

    }
}
```

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {

        HashMap<Integer, Integer> map = new HashMap<>();

        for(int i = 0; i < nums.length; i++){
            int num = nums[i];
            int comp = target - num;
            if(map.containsKey(comp)){
                return new int[] {map.get(comp), i};
            }
            map.put(num,i);
        }

        return new int[] {};
    }
}
```

PROBLEM TWO: VALID ANAGRAM

Given two strings **s** and **t**, return **true** if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Example 3:

Input: s = "", t = ""

Output: true

PROBLEM TWO: VALID ANAGRAM

Given two strings **s** and **t**, return **true** if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

** Hint 1:

How might you be able to use a HashMap (you can also use an array)?

Also think about key characteristics of anagrams - is there anything you can check that will automatically return False?

PROBLEM TWO: VALID ANAGRAM

Given two strings **s** and **t**, return **true** if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

** Hint 2 (simplicity):

Code too long? You could also consider helper functions, like .sort(). How might that affect runtime?

Q: Why do we use a vector instead of a map here? What situations can a map be replaced with a vector?

Possible C++ Solution:

```cpp
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) return false;

        vector<int> charCount(26, 0);
        for (int i = 0; i < s.size(); ++i) {
            ++charCount[s[i] - 'a'];
            --charCount[t[i] - 'a'];
        }
        return find_if(
                    charCount.begin(),
                    charCount.end(),
                    [](const int& c) { return c != 0; }
                ) == charCount.end();
    }
};
```

PROBLEM THREE: Longest Consec. Seq.

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in O(n) time.

Example 1:

Input: nums = [100,4,200,1,3,2]

Output: 4

Example 2:

Input: nums = [1,0,1,2]

Output: 3

PROBLEM THREE: Longest Consec. Seq.

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in O(n) time.

Hint:

Think back to what we learned earlier about the difference btw a HashSet and HashMap

Solution using a hashset

Python | Java | C++ | JavaScript | C# | Go | Kotlin | Swift

```python
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        numSet = set(nums)
        longest = 0

        for num in numSet:
            if (num - 1) not in numSet:
                length = 1
                while (num + length) in numSet:
                    length += 1
                longest = max(length, longest)
        return longest
```

TAKEAWAYS

01

Hashmaps: set of key-value pairs where there are no duplicate keys

02

Hashmaps allow you to look up keys without searching (because it is all unique)

03

Runtime: It’s important to optimize our code to run faster

04

05

06

ATTENDANCE FORM!

BRING YOUR FRIENDS!

Anyone with experience in coding, or who is taking data structures right now is more than welcome

NEXT MEETING in Schermerhorn 608 or Uris 330!